Define, together with the research team, the technologies, languages of programming, frameworks and libraries required for development, training, Optimizing and deploying AI models on mobile devices and augmented reality glasses.

AI model deployment on mobile devices and AR glasses requires Python with TensorFlow, PyTorch, or Keras for development and training, C++ for optimization, platform-specific languages (C# for Unity-based AR, Java/Kotlin for Android, Swift/Objective-C for iOS), mobile inference engines (TensorFlow Lite, PyTorch Mobile, or Core ML), and optimization techniques including quantization, pruning, and lightweight architectures such as MobileNet or SqueezeNet.

Abstract

AI model deployment on mobile devices and augmented reality glasses requires a multi-layered technology stack spanning development, optimization, and deployment phases. Python serves as the primary language for model development with TensorFlow, PyTorch, and Keras, while C++ is employed for optimization tasks. Platform-specific deployment necessitates different languages: C# for Unity-based AR applications, Java/Kotlin for Android, and Swift/Objective-C for iOS. Development frameworks include TensorFlow, PyTorch, and MXNet for model training, Unity 3D with AR Foundation and ARCore SDK for AR applications, and deployment relies on mobile inference engines including TensorFlow Lite, PyTorch Mobile, and Apple's Core ML for iOS. Optimization techniques are critical, with quantization achieving model size reductions up to 74% and inference speed improvements of 44%, while lightweight architectures like SqueezeNet achieve models over 500 times smaller than traditional architectures.

Technology selection depends on deployment context and must address substantial constraints including limited computational resources, memory, and battery consumption. Platform compatibility requires different approaches: Android deployments utilize TensorFlow Lite and PyTorch Mobile, iOS deployments use Core ML, and AR applications require Unity-based frameworks with specialized plugins. Performance requirements demand predictions within milliseconds while maintaining accuracy, necessitating careful trade-offs between model size and performance. The evidence indicates that successful deployment requires hierarchical technology selection: choosing target platform, selecting development frameworks based on application type, implementing platform-specific deployment technologies, and applying optimization techniques appropriate to model architecture and resource constraints.

Paper search

We performed a semantic search across over 138 million academic papers from the Elicit search engine, which includes all of Semantic Scholar and OpenAlex.

We ran this query: ”Define, together with the research team, the technologies, languages of programming, frameworks and libraries required for development, training, Optimizing and deploying AI models on mobile devices and augmented reality glasses.”

The search returned 50 total results from Elicit.

We retrieved 50 papers most relevant to the query for screening.

Screening

We screened in sources based on their abstracts that met these criteria:

- **Mobile/AR Platform Focus**: Does the study focus on AI/ML model development, training, optimization, or deployment specifically for mobile devices (smartphones, tablets) or augmented reality glasses?
- **Technology Components**: Does the research describe or evaluate programming languages, frameworks, libraries, or development tools used for mobile/AR AI applications?
- **Technical Implementation**: Does the study report on technical architectures, software stacks, or development pipelines for mobile/AR AI systems?
- **Optimization Techniques**: Does the research examine performance optimization techniques, model compression, or edge computing solutions for AI on mobile/AR devices?
- **Publication Type**: Is this a technical report, conference paper, or peer-reviewed article from computer science, engineering, or related fields?
- **Platform Relevance**: Does the study include mobile or AR components (rather than focusing solely on desktop or server-based AI applications)?
- **Practical Implementation**: Does the research include practical implementation details or technological specifications (rather than being limited to theoretical AI concepts only)?
- **Software Focus**: Does the study include software/programming aspects (rather than focusing exclusively on hardware design or manufacturing of mobile/AR devices)?
- **Domain Relevance**: Is the study relevant to mobile or AR platforms (rather than focusing solely on other domains like IoT, automotive, or robotics without mobile/AR relevance)?
- **Academic Rigor**: Does the study contain empirical data or technical validation (rather than being an opinion piece, editorial, or non-peer-reviewed blog post)?

We considered all screening questions together and made a holistic judgement about whether to screen in each paper.

Data extraction

We asked a large language model to extract each data column below from each paper. We gave the model the extraction instructions shown below for each column.

- **Target Platforms**:

  Extract all specific mobile devices and augmented reality glasses mentioned as target platforms for AI model deployment, including:

- Device models and specifications (e.g., Xiaomi Redmi Note 8 Pro, HoloLens, Google Lens, Jio Glass)
- Hardware characteristics relevant to AI deployment (CPU, GPU, memory, power constraints)
- Operating systems and versions
- Hardware architectures (FPGA, ASIC, mobile GPUs) used for AI processing

- **Programming Languages**:

  Extract all programming languages mentioned for developing, training, optimizing, or deploying AI models on mobile devices and AR glasses. Include the specific use case or stage where each language is applied (e.g., Python for model development, C++ for optimization, Java/Kotlin for Android deployment).

- **Development Frameworks**:

Extract frameworks and libraries used for developing and training AI models intended for mobile/AR deployment, including:

- Deep learning frameworks (e.g., TensorFlow, PyTorch, MXNet)
- Mobile-specific development frameworks (e.g., Unity3D for AR applications)
- Training platforms and tools
- Model development environments
- Specify what each framework/library is used for in the development pipeline

- **Deployment Technologies**:

  Extract frameworks, libraries, and technologies specifically used for deploying AI models on mobile devices and AR glasses, including:

- Mobile inference engines (e.g., TensorFlow Lite, MACE, Paddle-mobile)
- AR-specific deployment platforms
- Model conversion and adaptation tools
- Runtime environments for mobile AI
- Integration technologies for embedding AI into mobile/AR applications

- **Optimization Techniques**:

  Extract all optimization techniques mentioned for making AI models suitable for mobile devices and AR glasses, including:

- Model compression methods (quantization, pruning, knowledge distillation)
- Architecture optimizations (depthwise separable convolutions, MobileNet variants)
- Hardware-aware optimizations
- Graph optimization techniques (operator fusion, node reordering)
- Memory and computational optimizations
- Include quantitative results where available (e.g., model size reduction percentages, inference speed improvements)

- **Performance Metrics**:

  Extract performance requirements and evaluation metrics specific to AI models on mobile devices and AR glasses, including:

- Inference time and latency requirements
- Model size and memory usage constraints
- Battery consumption considerations
- Accuracy/performance trade-offs
- Real-time processing requirements
- SWAP (Size, Weight, Area, Power) considerations
- Specific benchmark results for mobile/AR AI applications

- **Implementation Challenges**:

  Extract challenges and obstacles encountered when developing, training, optimizing, or deploying AI models on mobile devices and AR glasses, along with proposed solutions, including:

- Resource constraints (computational, memory, power)
- Model conversion and adaptation issues

- Platform compatibility problems
- Performance vs. accuracy trade-offs
- Real-time processing challenges
- Data quality and training dataset limitations
- Specific solutions or workarounds proposed for each challenge

Results

Characteristics of included studies

Study	Full text retrieved?	Primary Focus	Research Type
Hamid Tabani et al., 2021	No	Challenges and obstacles in deploying deep learning models on mobile devices	Review of challenges and practical solutions
Gokila Deepa G et al., 2024	No	Real-time image processing in AR with low-latency Edge AI	Performance optimization study
V. Chornenkyi et al., 2024	No	ML models and optimization strategies for mobile device classification	Optimization techniques evaluation
Mathieu Febvay et al., 2020	No	Low-level optimizations for mobile deep learning inference frameworks	Framework performance benchmarking
R. Pierdicca et al., 2024	No	DeepReality framework for AI-based AR applications	Framework development and validation
Chunjie Luo et al., 2020	No	Comparison and benchmarking of AI models and frameworks on mobile devices	Benchmark suite development
Yang Wang et al., 2022	Yes	Key technologies for deep learning models targeting mobile terminals	Review of optimization and frameworks
Yash Prakash Gupta et al., 2022	Yes	Deep learning model optimization in AR applications	AR application optimization study
Dheeraj Vaddepally et al., 2025	Yes	Lightweight deep learning for resource-constrained devices	Review of lightweight model techniques
Yunbin Deng et al., 2019	Yes	Deep learning on mobile devices	Comprehensive review of mobile deep learning

The studies span from 2019 to 2025, with six studies available only as abstracts and four with full text access. Re-

search approaches include comprehensive reviews, framework development, optimization studies, and performance benchmarking. While most studies focus broadly on mobile devices, several specifically address augmented reality applications.

Target Platforms and Hardware Architectures

Device Category	Specific Devices/Hardware	Operating Systems	Hardware Architectures	Sources
Mobile devices	Xiaomi Redmi Note 8 Pro, iPhone 7, Phone 7 plus, Samsung Galaxy S8	iOS, Android	CPUs, GPUs, NVIDIA GPUs with CUDA, Qualcomm Snapdragon chip	Multiple sources
AR glasses	HoloLens, Google Lens, Jio Glass (75 grams)	Not specified	Not specified	Gupta et al., 2022
Edge devices	Resource-constrained edge devices including smartphones and AR glasses	Not specified	Not specified	Deepa et al., 2024
Specialized hardware	Not specified	Not specified	FPGAs, ASICs	Deng et al., 2019
IoT systems	Mobile devices and IoT systems	Android and iOS	Limited memory, power, and computational resources	Vaddepally et al., 2025

The target platforms span a range of devices from smartphones to specialized AR glasses. Hardware architectures include traditional CPUs and GPUs, with NVIDIA GPUs and CUDA platforms specifically mentioned for machine learning tasks. Mobile system-on-chip solutions like Qualcomm Snapdragon are highlighted as notable platforms for smartphones. Specialized hardware architectures including FPGAs and ASICs are discussed for mobile AI processing. Resource constraints are a common theme, with devices characterized by limited memory, power, and computational resources.

Programming Languages for AI Model Development and Deployment

Language	Use Cases	Frameworks/Platforms	Sources
Python	Model development with TensorFlow, Keras, and PyTorch, building APIs with FastAPI, used with CUDA for machine learning tasks, basis for PyTorch	TensorFlow, Keras, PyTorch, FastAPI, CUDA	Wang et al., 2022; Gupta et al., 2022; Deng et al., 2019
C++	Optimization and deployment with TensorFlow, core language for Caffe, used with CUDA for machine learning	TensorFlow, Caffe, CUDA	Wang et al., 2022; Deng et al., 2019
C#	Unity development and integration	Unity 3D	Pierdicca et al., 2024
Java	Machine learning, implied for Android deployment	CNNDroid supporting Caffe, Torch, and Theano on Android	Deng et al., 2019; Vaddepally et al., 2025
Kotlin	Implied for Android deployment	Android frameworks	Vaddepally et al., 2025
Swift/Objective-C	Implied for iOS deployment	iOS frameworks	Vaddepally et al., 2025
C	Used with CUDA for machine learning tasks	CUDA	Deng et al., 2019
FORTRAN	Used with CUDA for machine learning tasks	CUDA	Deng et al., 2019
Matlab	Used in machine learning	Various	Deng et al., 2019
R	Used in machine learning	Various	Deng et al., 2019
Lua	Programming language for Torch, related to PyTorch	Torch	Deng et al., 2019
CUDA	Used for machine learning tasks on NVIDIA GPUs	NVIDIA GPUs	Deng et al., 2019

Python emerges as the predominant language for AI model development and training, particularly with major frameworks like TensorFlow, PyTorch, and Keras. C++ is commonly used for optimization and deployment tasks, serving as the core language for frameworks like Caffe. For mobile-specific development, platform-dependent languages are employed: C# for Unity-based AR applications, Java/Kotlin for Android deployment, and Swift/Objective-C for iOS. CUDA serves both as a programming language and platform for GPU-accelerated machine learning, supporting multiple languages including Python, C, C++, and FORTRAN.

Development Frameworks and Libraries

Framework Category	Specific Frameworks/Libraries	Primary Use	Platform Support	Sources
Deep learning frameworks	TensorFlow, PyTorch, Caffe/Caffe2, Keras, MXNet	Model development and training	Cross-platform	Multiple sources
AR development frameworks	Unity 3D, AR Foundation, ARCore SDK	Creating AR applications	iOS and Android	Pierdicca et al., 2024; Gupta et al., 2022
Model inference engines	Barracuda	Model inference in Unity	Unity 3D	Pierdicca et al., 2024
Model optimization tools	TensorRT	Optimizing model performance	NVIDIA platforms	Gupta et al., 2022
Cloud computing platforms	AWS EC2	Deploying API and offloading computations to cloud GPU	Cloud	Gupta et al., 2022
API development	FastAPI	Building high-performance APIs	Cross-platform	Gupta et al., 2022
Computer vision	OpenCV	Processing multimedia, image processing, face detection	Cross-platform	Gupta et al., 2022
Model conversion tools	MMdnn, Detectron	Model conversion, object detection	Cross-platform	Deng et al., 2019
Mobile-specific frameworks	Mobile AI Compute Engine (MACE), Paddle-mobile, CNNDroid, DeepX	Mobile deep learning inference	Mobile platforms	Deng et al., 2019
High-level APIs	Keras	Simplifying TensorFlow, CNTK, Theano	Cross-platform	Deng et al., 2019

TensorFlow and PyTorch are highlighted as the primary deep learning frameworks, with TensorFlow requiring good knowledge of machine learning and software engineering, while PyTorch is popular among academic researchers. Keras serves as a high-level API that simplifies the use of lower-level frameworks. For AR applications, Unity 3D is the dominant development platform, integrated with AR Foundation and ARCore SDK for mobile AR functionality. MXNet is described as flexible and efficient with cross-platform capabilities. Mobile-specific frameworks like MACE and Paddle-mobile are designed specifically for mobile deep learning inference.

Deployment Technologies for Mobile and AR Platforms

Technology Category	Specific Technologies	Functionality	Platform Support	Sources
Mobile inference engines	TensorFlow Lite, Caffe2, PyTorch Mobile, FeatherCNN, MACE, Paddle-mobile	Optimized inference on mobile devices	iOS and Android	Multiple sources
AR-specific platforms	AR Foundation, ARCore SDK	AR functionality and multimedia extraction	Android	Pierdicca et al., 2024; Gupta et al., 2022
Inference engines	Barracuda inference engine	Model inference in Unity applications	Unity 3D	Pierdicca et al., 2024
Runtime environments	TensorFlow Lite, Caffe2	Running models on mobile devices	iOS and Android	Multiple sources
iOS frameworks	Apple's Core ML framework	AI deployment on iOS devices	iOS	Vaddepally et al., 2025
Integration platforms	Unity 3D, OpenCV API, FastAPI	Embedding AI into mobile/AR applications	Cross-platform	Multiple sources
Model conversion tools	MMdnn	Model conversion and adaptation	Cross-platform	Deng et al., 2019
Benchmarking tools	TensorFlow Lite Benchmark, PyTorch Mobile, MLPerf	Performance evaluation and optimization	Mobile platforms	Vaddepally et al., 2025

TensorFlow Lite emerges as the most widely adopted mobile inference engine, mentioned across multiple studies as a lightweight solution for deploying models on mobile devices. PyTorch Mobile provides an alternative for PyTorch-based models. Platform-specific solutions include Apple's Core ML framework for iOS devices, while AR Foundation and ARCore SDK provide AR-specific functionality. Four popular open-source deep learning inference frameworks were benchmarked, though specific names were not provided in all abstracts. The DeepReality framework specifically integrates deep learning models into Unity through AR Foundation and Barracuda inference engine.

Optimization Techniques and Quantitative Results

Optimization Category	Specific Techniques	Quantitative Results	Application Context	Sources
Model compression: Quantization	Quantization, post-training quantization, local quantization, exponential quantization, reducing precision from 32 bits to 16 bits or less, binarization	Model size reduction up to 74%, inference speed improvement of 44%	Vision Transformers and mobile CNNs	Chornenkyi et al., 2024; Wang et al., 2022; Deng et al., 2019
Model compression: Pruning	Multi-granularity pruning, pruning with batch normalization factor and filter correlation, joint dynamic pruning, pruning based on cross-entropy, operator fusion, node reordering	Not specified	Various neural networks	Wang et al., 2022; Chornenkyi et al., 2024; Vaddepally et al., 2025
Model compression: Knowledge distillation	Multi-module feature training, self-distillation	Not specified	Model training optimization	Wang et al., 2022; Vaddepally et al., 2025
Architecture optimization	Depthwise separable convolutions (MobileNet), group-by-point convolution and channel rearrangement (ShuffleNet), replacing 3x3 filters with 1x1 filters, decreasing input channels	SqueezeNet achieved 0.5MB size, over 500 times smaller than AlexNet	Lightweight network design	Wang et al., 2022; Deng et al., 2019
Lightweight models	MobileNet, EfficientNet, SqueezeNet, MobileNetV2, MnasNet, ShuffleNet	Latency reduction of 30%, accuracy improvement of 10%	Resource-constrained edge devices	Multiple sources

Optimization Category	Specific Techniques	Quantitative Results	Application Context	Sources
Training optimization	Adaptive learning rate scheduling, data augmentation	Accuracy improvement of 10%	AR applications	Deepa et al., 2024
Hardware-aware optimization	TensorFlow Lite, FeatherCNN with layer fusion, co-designing algorithms and hardware for ASIC chips	Not specified	Mobile deployment	Wang et al., 2022; Deng et al., 2019
Graph optimization	Operator fusion, pruning, node reordering, layer fusion	Reduced computational complexity	Mobile inference	Chornenkyi et al., 2024; Wang et al., 2022
Neural Architecture Search	Automated neural network architectures	Not specified	Hardware-specific optimization	Vaddepally et al., 2025
Memory optimization	Weight sharing	Not specified	Parameter reduction	Vaddepally et al., 2025
Binary optimization	XNOR network with binary weights	Same accuracy as full-precision AlexNet	Inference speed optimization	Deng et al., 2019

Quantization emerges as one of the most effective optimization strategies for mobile environments, with studies showing model size reductions up to 74% and inference speed improvements of 44% for Vision Transformers. Architecture optimizations through lightweight models demonstrate significant efficiency gains, with SqueezeNet achieving a model size over 500 times smaller than AlexNet while maintaining comparable accuracy. The XNOR network achieved the same accuracy as full-precision AlexNet using binary weights. Lightweight CNNs like MobileNet and EfficientNet achieved latency reductions of 30% and accuracy improvements of 10% in AR scenarios. Hardware-aware optimizations are essential, with the need for conversion, adaptation, and sometimes full retraining. Graph optimization techniques including operator fusion, pruning, and node reordering reduce computational complexity.

Performance Requirements and Evaluation Metrics

Metric Category	Specific Metrics	Requirements/Constraints	Application Context	Sources
Latency and inference time	Inference time, low latency, latency reduction, predictions within milliseconds, Media Retrieval Time, API Process Time, Client Time	Latency reduction of 30%, predictions should be made within milliseconds	Real-time AR applications, mobile inference	Multiple sources
Model size and memory	Model size, memory usage, model size reduction	Model size reduction up to 74%, SqueezeNet reduces model size by a factor of 50, lower memory requirements	Resource-constrained devices	Multiple sources
Battery and power	Battery consumption, power consumption, energy efficiency	Critical to prevent rapid battery drain	Mobile devices	Chornenkyi et al., 2024; Vaddepally et al., 2025; Deng et al., 2019
Accuracy and performance	Accuracy, accuracy improvement, balance between accuracy and latency	Accuracy improvement of 10%, finding balance between model size and accuracy	All applications	Multiple sources
Real-time processing	Real-time performance, quick response time	Essential for mobile devices, support real-time inference	AR and autonomous driving	Multiple sources
Computational efficiency	Inference speed improvement, CPU usage, Valid Images Per Second (VIPS), Valid FLOPs Per Second (VOPS)	Inference speed improvement up to 44%	Mobile and embedded devices	Chornenkyi et al., 2024; Luo et al., 2020
SWAP considerations	Size, Weight, Area, Power	Key consideration in hardware architecture decisions	Mobile hardware design	Deng et al., 2019
Frame rate	Frames per second, frame rate impact on latency	High enough for smooth usability, higher frame rates increase latency	AR applications	Deepa et al., 2024; Gupta et al., 2022

Benchmark metrics	Execution time, model size, speed/execution time	Various benchmarks for comparison	Performance evaluation	Pierdicca et al., 2024; Deng et al., 2019

Performance requirements emphasize the critical balance between multiple constraints. Models must provide predictions within milliseconds for real-time applications, with latency reductions of 30% demonstrated in AR scenarios. Model size constraints are substantial, with quantization achieving up to 74% size reduction and lightweight architectures like SqueezeNet reducing size by a factor of 50 compared to traditional models. Energy efficiency is critical to prevent rapid battery drain, making battery consumption a key consideration. Two unified metrics, Valid Images Per Second (VIPS) and Valid FLOPs Per Second (VOPS), were proposed for comparing and ranking AI capabilities of mobile devices. SWAP (Size, Weight, Area, Power) considerations are key factors in hardware architecture decisions. Performance benchmarks consider accuracy, model size, and speed/execution time.

Implementation Challenges and Solutions

Challenge Category	Specific Challenges	Proposed Solutions	Quantitative Impact	Sources
Resource constraints	Limited computational resources, limited memory, battery consumption, limited computing power, limited power	Quantization (reduces model size by up to 74%, improves inference speed by 44%), graph optimization techniques, TensorFlow Lite, FeatherCNN, lightweight models, offloading computations to cloud GPUs, pruning, optimization	Model size reduction up to 74%, inference speed improvement of 44%	Multiple sources
Real-time processing	Need for real-time performance, significant challenges for real-time AR tasks, real-time computation requirements, predictions within milliseconds despite computational overhead	Using edge computing and optimizing API processing times, automated optimization techniques, hardware-software co-design	Not specified	Multiple sources

Challenge Category	Specific Challenges	Proposed Solutions	Quantitative Impact	Sources
Performance vs accuracy trade-offs	Balance between accuracy and latency, too much pruning leads to poor performance, compression reduces model performance, reducing model size can lead to accuracy loss	Iterative pruning and training process, optimizing frame rates, finding balance between model size and accuracy	Not specified	Multiple sources
Model conversion and adaptation	Hardware-aware optimizations required, conversion to other formats or using third-party frameworks, model conversion steps impact performance, optimizing communication between local and cloud frameworks, platform-specific optimization	Conversion, adaptation, and retraining, low-level software implementations	Not specified	Multiple sources
Platform compatibility	Variety of hardware platforms with heterogeneous computing elements, frameworks run at different speeds, Caffe lacks support for recursive networks, differences in system APIs and frameworks between Android and iOS, hardware support differs between Android and iOS	TensorFlow Lite for security and performance, using ARCore SDK with deep learning models, platform-specific optimization required, ensuring compatibility across different hardware architectures	Not specified	Multiple sources

Challenge Category	Specific Challenges	Proposed Solutions	Quantitative Impact	Sources
Data quality and training	Low-quality data, small training datasets, variability in testing data, need for large amounts of data	Algorithmic optimizations (quantization, pruning, compression), hardware co-design approaches	Not specified	Deng et al., 2019; Gupta et al., 2022
Framework implementation	Difficulties implementing frameworks inside mobile applications, marker identification and content creation are time-consuming	Simplified integration through frameworks like DeepReality	Not specified	Febvay et al., 2020; Pierdicca et al., 2024

Hardware-aware optimizations are required for converting models to run on mobile devices, often necessitating conversion to other formats or using third-party frameworks. Limited computational resources, memory, and battery consumption present substantial challenges, addressed through quantization that achieves model size reductions up to 74% and inference speed improvements of 44%. Real-time processing requirements pose significant challenges, particularly for augmented reality applications, with solutions including edge computing and optimized API processing. Performance versus accuracy trade-offs are critical, as aggressive compression can reduce model performance and excessive pruning leads to poor accuracy. Platform compatibility issues arise from heterogeneous hardware platforms and framework differences, with TensorFlow running slower than other frameworks in some cases and Caffe lacking support for certain network types. Data quality challenges include low-quality data, small training datasets, and variability in testing data, requiring algorithmic optimizations and hardware co-design approaches. Future advancements include automated optimization techniques and hardware-software co-design to improve performance and efficiency.

Synthesis

The landscape of AI deployment on mobile devices and augmented reality glasses reveals a complex ecosystem requiring careful coordination of multiple technological components. The evidence demonstrates distinct optimization requirements for different deployment contexts, with quantitative results showing that quantization achieves model size reductions up to 74% and inference speed improvements of 44% in Vision Transformers, while lightweight architectures like SqueezeNet achieve model sizes over 500 times smaller than AlexNet. These optimizations are not universally applicable—studies finding substantial benefits from quantization focus on specific model architectures like Vision Transformers, while studies examining lightweight CNN designs like MobileNet and EfficientNet report different metrics of latency reduction (30%) and accuracy improvement (10%).

The apparent diversity in framework recommendations across studies reflects genuine differences in deployment contexts rather than contradictory findings. Studies emphasizing TensorFlow Lite typically focus on cross-platform deployment and standardized inference, while those highlighting platform-specific solutions like Apple's Core ML or Unity-based approaches with Barracuda address specialized requirements for iOS or AR applications respectively.

Both approaches are valid within their respective deployment scenarios.

The synthesis reveals that optimization effectiveness depends critically on the baseline model architecture. Quantization provides substantial benefits for complex models like Vision Transformers, achieving 74% size reduction, whereas architectural optimizations through lightweight design (as in SqueezeNet) can achieve even more dramatic reductions—over 500 times smaller than traditional architectures—but with different accuracy trade-offs. Studies reporting higher efficiency gains from lightweight models focus on designed-for-mobile architectures, while those emphasizing post-training quantization address the challenge of adapting existing models.

Platform compatibility challenges reflect fundamental differences in hardware capabilities and software ecosystems. Android deployments benefit from frameworks like TensorFlow Lite and PyTorch Mobile, with explicit support through CNNDroid for Caffe, Torch, and Theano models. iOS deployments utilize Core ML with different optimization requirements due to distinct system APIs and hardware support. AR applications introduce additional complexity, requiring specialized frameworks like Unity 3D with AR Foundation and ARCore SDK, where cloud offloading strategies can address computational limitations at the cost of increased latency in network communication.

The evidence indicates that successful deployment requires a hierarchical approach to technology selection: first determining the target platform (Android, iOS, or AR glasses), then selecting appropriate development frameworks (TensorFlow/PyTorch for model development, Unity for AR applications), followed by platform-specific deployment technologies (TensorFlow Lite, PyTorch Mobile, or Core ML), and finally applying appropriate optimization techniques based on model architecture and resource constraints. Studies examining multiple frameworks consistently find that low-level software implementations, model conversion steps, and framework parameters significantly impact both performance and accuracy, suggesting that framework selection cannot be separated from optimization strategy.

References

Chunjie Luo, Xiwen He, Jianfeng Zhan, Lei Wang, Wanling Gao, and Jiahui Dai. "Comparison and Benchmarking of AI Models and Frameworks on Mobile Devices." *arXiv.org*, 2020.

Dheeraj Vaddepally. "Lightweight Deep Learning for Resource-Constrained Devices." *International Journal for Sciences and Technology*, 2025.

Gokila Deepa G, Gomathi S, A. S, Sujitha R, Sundarrajan M, and Mani Deepak Choudhry. "Optimizing Real-Time Image Processing in Augmented Reality with Low-Latency Edge AI." *Software and Compilers for Embedded Systems*, 2024.

Hamid Tabani, Ajay Balasubramaniam, E. Arani, and Bahram Zonooz. "Challenges and Obstacles Towards Deploying Deep Learning Models on Mobile Devices." *arXiv.org*, 2021.

Mathieu Febvay. "Low-Level Optimizations for Faster Mobile Deep Learning Inference Frameworks." *ACM Multimedia*, 2020.

R. Pierdicca, Flavio Tonetto, M. Paolanti, M. Mameli, Riccardo Rosati, and P. Zingaretti. "DeepReality: An Open Source Framework to Develop AI-Based Augmented Reality Applications." *Expert Systems with Applications*, 2024.

V. Chornenkyi, and I. Kazymyra. "ML MODELS AND OPTIMIZATION STRATEGIES FOR ENHANCING THE PERFORMANCE OF CLASSIFICATION ON MOBILE DEVICES." *Ukrainian Journal of Information Technology*, 2024.

Yang Wang, Bohai Yi, and Kexin Yu. "Exploration and Research about Key Technologies Concerning Deep Learning Models Targeting Mobile Terminals." *Journal of Physics: Conference Series*, 2022.

Yash Prakash Gupta, Mukul, and N. Gupta. "Deep Learning Model Based Multimedia Retrieval and Its Optimization in Augmented Reality Applications." *Multimedia Tools and Applications*, 2022.

Yunbin Deng. "Deep Learning on Mobile Devices: A Review." *Defense + Commercial Sensing*, 2019.